Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence

Data Structures and Algorithm Analysis (Fall-2026)

LAB No. 4

Objective of Lab No. 4:

After performing Lab 04, students will be able to:

- Implement array insertion at the beginning, middle, end, and a specified position using Python lists.
- Implement array deletion from the beginning, middle, end, and a specified position.
- Understand how element shifting affects insertion and deletion performance.
- Implement array modification using direct indexed access.
- Implement Linear Search for locating values in unsorted data.
- Implement Binary Search for locating values in sorted data.
- Understand the precondition that Binary Search requires sorted data.
- Analyze and compare the time and auxiliary space complexity of Linear Search and Binary Search.
- Develop the ability to select an appropriate searching technique based on the nature of the data.
- Measure execution time using time.perf_counter().
- Analyze time and auxiliary space complexity using Big-O notation.

Important Note:

Python lists are used as array-like structures for this laboratory. Although Python lists are dynamically sized, the exercises focus on indexed positions, insertion, deletion, shifting, and modification—the core ideas required for array operation analysis.

Lab Tasks:

Task 01 — Array Insertion at the End

Insertion at the end adds a new element after the last existing element. No existing element normally needs to shift.

Solution:

```
import time

marks = [70, 75, 80, 85, 90]

start_time = time.perf_counter()

marks.append(95)

end_time = time.perf_counter()
```

```
print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1) amortized")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(1) amortized

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: append() normally places the new element at the end without shifting existing elements. Python may occasionally resize the underlying list, but over many operations the average cost per append remains constant.

Task 02 — Array Insertion at the Beginning

Insertion at the beginning requires the existing elements to move one position to the right to create space for the new element.

Solution:

```
import time

marks = [70, 75, 80, 85, 90]

start_time = time.perf_counter()

marks.insert(0, 65)

end_time = time.perf_counter()

print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(n)

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: The insertion position is index 0. Therefore, all existing n elements may shift one position to the right. As the number of elements grows, the number of required shifts grows linearly.

Task 03 — Array Insertion at a Middle Position

Insertion in the middle creates space at the specified index. Elements from that position onward shift to the right.

Solution:

```
import time

marks = [70, 75, 80, 85, 90]
```

```
start_time = time.perf_counter()

marks.insert(2, 78)

end_time = time.perf_counter()

print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n) worst case")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(n) worst case

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: To insert 78 at index 2, the elements previously at index 2 and after it may shift one position to the right. The number of shifted elements can grow with n.

Task 04 — Array Insertion at a Specified Position

This task accepts a position and inserts a value at that position. It demonstrates that the insertion cost depends on where the new element is placed.

Solution:

```
import time

numbers = [10, 20, 30, 40, 50]

position = 3
value = 99

start_time = time.perf_counter()

numbers.insert(position, value)

end_time = time.perf_counter()

print("Updated Array:", numbers)
print("Inserted Value:", value)
print("Position:", position)
print("Execution Time:", end_time - start_time, "seconds")
print("Worst-Case Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(n) worst case

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: Insertion at a specified position may require shifting all elements from that position to the end. In the worst case, the number of shifted elements is proportional to n.

Task 05 — Array Deletion from the End

Deletion from the end removes the final element. The remaining elements do not need to change position.

Solution:

```
import time

students = ["Ali", "Sara", "Ahmed", "Hina", "Usman"]

start_time = time.perf_counter()

removed = students.pop()

end_time = time.perf_counter()

print("Removed Element:", removed)
print("Updated Array:", students)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(1)

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: pop() without an index removes the last element. No remaining elements need to shift, so the operation requires constant time.

Task 06 — Array Deletion from the Beginning

Deleting the first element creates a gap at index 0. All remaining elements shift one position to the left.

Solution:

```
import time

marks = [65, 70, 75, 80, 85, 90]

start_time = time.perf_counter()

removed = marks.pop(0)

end_time = time.perf_counter()

print("Removed Element:", removed)
print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(n)

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: After deleting index 0, every remaining element may shift left by one position. The number of shifts grows linearly with the size of the array.

Task 07 — Array Deletion from a Middle Position

Deleting an element from the middle requires the elements after the deleted position to shift left.

Solution:

```
import time

marks = [65, 70, 75, 80, 85, 90]

start_time = time.perf_counter()

removed = marks.pop(2)

end_time = time.perf_counter()

print("Removed Element:", removed)
print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Worst-Case Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(n) worst case

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: The element at index 2 is removed and the elements after it shift left to fill the gap. The number of shifted elements can grow with n.

Task 08 — Array Deletion from a Specified Position

This task deletes an element using a specified index and shows how the cost depends on the position of the element.

Solution:

```
import time

numbers = [10, 20, 30, 40, 50, 60]

position = 3

start_time = time.perf_counter()

removed = numbers.pop(position)

end_time = time.perf_counter()

print("Removed Element:", removed)
print("Updated Array:", numbers)
```

```
print("Deleted Position:", position)
print("Execution Time:", end_time - start_time, "seconds")
print("Worst-Case Time Complexity: O(n)")
print("Auxiliary Space Complexity: O(1) auxiliary")
```

Time Complexity: O(n) worst case

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: The elements after the deleted position shift left. Deleting near the beginning generally requires more shifting than deleting near the end.

Task 09 — Array Modification at a Known Index

Modification replaces the value at a known index. The size and positions of the remaining elements do not change.

Solution:

```
import time

marks = [75, 82, 91, 68, 88]

start_time = time.perf_counter()

marks[2] = 95

end_time = time.perf_counter()

print("Updated Array:", marks)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1)")
print("Auxiliary Space Complexity: O(1)")
```

Time Complexity: O(1)

Auxiliary Space Complexity: O(1)

Complexity Analysis: The index is already known, so Python directly accesses the required position and replaces its value. No traversal or shifting is required.

Task 10 — Multiple Array Modifications

A fixed number of known-index updates remains constant time because the number of operations does not grow with the array size.

Solution:

```
import time

scores = [60, 65, 70, 75, 80, 85]

start_time = time.perf_counter()
```

```
scores[1] = 90
scores[4] = 95

end_time = time.perf_counter()

print("Updated Array:", scores)
print("Execution Time:", end_time - start_time, "seconds")
print("Time Complexity: O(1)")
print("Auxiliary Space Complexity: O(1)")
```

Time Complexity: O(1)

Auxiliary Space Complexity: O(1)

Complexity Analysis: Only two direct indexed updates are performed. Because the number of updates is fixed and does not increase with n, the overall operation is O(1).

Task 11: Update one element using a known index and another element after locating it.

Program:

```
products = ["Keyboard", "Mouse", "Monitor", "Printer"]


print("Original list:", products)


products[1] = "Wireless Mouse"

print("After index-based update:", products)


old_value = "Printer"

new_value = "Scanner"


for i in range(len(products)):

    if products[i] == old_value:

        products[i] = new_value

        break


print("After search-based update:", products)
```

Expected Output (example):

```
Original list: ['Keyboard', 'Mouse', 'Monitor', 'Printer']

After index-based update: ['Keyboard', 'Wireless Mouse', 'Monitor', 'Printer']

After search-based update: ['Keyboard', 'Wireless Mouse', 'Monitor', 'Scanner']
```

Explanation: The example distinguishes update cost from search cost.

Time Complexity: Known-index update: O(1). Search-based update: O(n) worst case.
Space Complexity: O(1) auxiliary space.
Justification: Updating at a known position is direct; locating an unknown value may require searching.

Task 12: Linear Search on Unsorted Data

Linear Search checks elements sequentially. It works on unsorted data because it makes no assumption about element order.

Objective: Search for a target value in an unsorted list.

Program:

```
numbers = [42, 17, 89, 5, 63, 28]

target = 63


found_index = -1


for i in range(len(numbers)):

    if numbers[i] == target:

        found_index = i

        break


print("List:", numbers)

print("Target:", target)


if found_index != -1:

    print("Target found at index:", found_index)

else:

    print("Target not found")
```

Expected Output (example):

```
List: [42, 17, 89, 5, 63, 28]

Target: 63

Target found at index: 4
```

Explanation: Linear Search is suitable when the collection is not sorted.

Time Complexity: Best O(1); average Θ(n); worst O(n).
Space Complexity: O(1) auxiliary space.
Justification: The algorithm may stop at the first element or may need to inspect all elements.

Task 13: Linear Search Returning All Matching Positions

Objective: Find every occurrence of a target value in an unsorted list.

Program:

```
scores = [55, 72, 55, 81, 90, 55, 68]

target = 55


positions = []


for i in range(len(scores)):

    if scores[i] == target:

        positions.append(i)


print("Scores:", scores)

print("Target:", target)


if positions:

    print("Target found at indexes:", positions)

else:

    print("Target not found")
```

Expected Output (example):

```
Scores: [55, 72, 55, 81, 90, 55, 68]

Target: 55

Target found at indexes: [0, 2, 5]
```

Explanation: This variation continues searching after a match.

Time Complexity: O(n)
Space Complexity: O(k), where k is the number of matching positions stored.
Justification: Every element must be checked because all occurrences are required.

Task 14: Binary Search on Sorted Data

Binary Search compares the target with the middle element of a sorted list and repeatedly eliminates approximately half of the remaining search space.

Objective: Search for a target in a sorted list using low, high, and mid indexes.

Program:

```
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90]

target = 70


beg= 0

end = len(numbers) - 1

found_index = -1


while beg <= end:

    mid = (beg + high) // 2


    if numbers[mid] == target:

        found_index = mid

        break

    elif numbers[mid] < target:

        beg = mid + 1

    else:

        end = mid - 1


print("Sorted list:", numbers)

print("Target:", target)
```

```
if found_index != -1:
    print("Target found at index:", found_index)
else:
    print("Target not found")
```

Expected Output (example):

```
Sorted list: [10, 20, 30, 40, 50, 60, 70, 80, 90]
Target: 70
Target found at index: 6
```

Explanation: The algorithm is valid only because the input is sorted.

Time Complexity: Best O(1); average/worst O(log n).
Space Complexity: O(1) auxiliary space.
Justification: Each unsuccessful comparison removes about half of the remaining search range.

Task 15: Binary Search Using a Function

Objective: Create a reusable Binary Search function and test found and not-found cases.

Program:

```
def binary_search(arr, target):
    beg = 0
    end = len(arr) - 1

    while beg <= end:
        mid = (beg + end) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            beg = mid + 1
        else:
            end = mid - 1

    return -1
```

```
student_ids = [1001, 1005, 1010, 1018, 1025, 1031, 1040]


target1 = 1018

target2 = 1020


print("Student IDs:", student_ids)

print(target1, "found at index:", binary_search(student_ids, target1))

print(target2, "search result:", binary_search(student_ids, target2))
```

Expected Output (example):

```
Student IDs: [1001, 1005, 1010, 1018, 1025, 1031, 1040]

1018 found at index: 3

1020 search result: -1
```

Explanation: A function allows the same algorithm to be reused on multiple sorted datasets.

Time Complexity: O(log n)
Space Complexity: O(1) auxiliary space.
Justification: The active interval is halved until the target is found or no interval remains.

Task 16: Compare Linear and Binary Search Complexity

Linear Search works on unsorted or sorted data but progresses one element at a time. Binary Search is more efficient for large datasets when sorted order is available.

Objective: Count how many comparisons Linear Search and Binary Search perform for the same target.

Program:

```
def linear_search_with_count(arr, target):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == target:
            return i, comparisons
```

```
    return -1, comparisons



def binary_search_with_count(arr, target):
    low = 0
    high = len(arr) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if arr[mid] == target:
            return mid, comparisons
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1, comparisons



numbers = list(range(1, 101))
target = 97

linear_result, linear_count = linear_search_with_count(numbers, target)
binary_result, binary_count = binary_search_with_count(numbers, target)

print("Target:", target)
```

```
print("Linear Search index:", linear_result)

print("Linear Search comparisons:", linear_count)


print("\nBinary Search index:", binary_result)

print("Binary Search comparisons:", binary_count)
```

Expected Output (example):

```
Target: 97

Linear Search index: 96

Linear Search comparisons: 97


Binary Search index: 96

Binary Search comparisons: considerably fewer than Linear Search
```

Explanation: Exact comparison counts can vary by target position, but asymptotic growth remains linear versus logarithmic.

Time Complexity: Linear Search O(n); Binary Search O(log n).
Space Complexity: O(1) auxiliary space for both iterative implementations.
Justification: Linear Search progresses sequentially, while Binary Search repeatedly halves the active interval.

Task 17: Compare Observed Execution Time

Objective: Measure observed execution time for both search algorithms on the same sorted dataset.

Program:

```
import time


def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1



numbers = list(range(1, 100001))
target = 99999

start = time.perf_counter()
linear_result = linear_search(numbers, target)
linear_time = time.perf_counter() - start

start = time.perf_counter()
binary_result = binary_search(numbers, target)
binary_time = time.perf_counter() - start
```

```
print("Linear Search result:", linear_result)

print("Linear Search time:", linear_time)


print("\nBinary Search result:", binary_result)

print("Binary Search time:", binary_time)
```

Expected Output (example):

```
Linear Search result: 99998

Linear Search time: machine-dependent


Binary Search result: 99998

Binary Search time: machine-dependent
```

Explanation: The target is near the end to make the sequential cost of Linear Search more visible.

Time Complexity: Linear Search O(n); Binary Search O(log n).
Space Complexity: O(1) auxiliary space for both iterative implementations.
Justification: Measured times depend on the machine and runtime environment. The important observation is the growth trend as input size increases.

Summary Comparison

Operation / Algorithm	Data Requirement	Best Case	Average Case	Worst Case
Direct list indexing	Known valid index	O(1)	O(1)	O(1)
List traversal	Any list	O(n)	O(n)	O(n)
Insert/delete at beginning or middle	Array/list	O(n)	O(n)	O(n)
Update at known index	Known valid index	O(1)	O(1)	O(1)
Linear Search	Sorted or unsorted	O(1)	Θ(n)	O(n)
Binary Search	Sorted data	O(1)	O(log n)	O(log n)

Aror University of Art, Architecture, Design & Heritage Sukkur.

Lab Exercise:

Implement the following tasks. For each task, measure execution time using time.perf_counter(), identify the time complexity, state the auxiliary space complexity, and justify your answer based on direct access or element shifting.

- Task 1: Create a list of 10 student marks. Insert a new mark at the end and explain why this operation is typically O(1) amortized.
- Task 2: Create a list of 10 student marks. Insert a new mark at index 0. Explain how many existing elements may shift and why the operation is O(n).
- Task 3: Create a list of 10 student marks. Insert a new mark at the middle index. Measure the operation and explain the relationship between the insertion position and the number of shifted elements.
- Task 4: Write a program that inserts a value at a user-specified valid index. Test the insertion near the beginning, middle, and end, and compare the observed execution behavior.
- Task 5: Create a list of student names and delete the last element. Explain why no remaining element needs to shift.
- Task 6: Create a list of student names and delete the first element. Explain why the remaining elements shift and determine the time complexity.
- Task 7: Delete an element from the middle of a list of at least 10 values. Count or explain the number of elements that must shift after deletion.
- Task 8: Write a program that deletes an element from a specified index. Test deletion near the beginning, middle, and end and compare the expected complexities.
- Task 9: Create a list containing marks of 10 students. Update three specified indexes and determine the time and auxiliary space complexity.
- Task 10: Update every element in a list using a loop. Compare its complexity with updating a fixed number of known indexes.

Scenario-Based: Daily Temperature Monitoring

Task 11: A weather monitoring application stores the daily temperatures of a week in a list. Display all temperatures, calculate the highest and lowest temperature, and calculate the average.

Requirements:

- Store at least 7 values.
- Use loops for maximum, minimum, and total calculation.
- Display meaningful labels.
- Analyze time and auxiliary space complexity.

Complexity Analysis: Explain why the overall running time is O(n).

Scenario-Based: Student Attendance List

Task 12: A classroom application maintains a list of student names. A new student joins, one student leaves, and one student's name must be corrected. Implement all three operations.

Requirements:

- Insert the new student at a specified position.
- Delete the student who leaves.
- Update the incorrect name.
- Display the final attendance list.
- Explain the complexity of locating a student when only the name is known.

Complexity Analysis: Separate the complexity of searching for a name from insertion/deletion/update at a known index.

Linear Search on Unsorted Data

Simple Linear Search

Task 13: Create an unsorted list of at least 12 integers and search for a user-defined target using Linear Search.

Requirements:

- Do not use index().
- Display the index when found.
- Display a not-found message otherwise.
- Test one found and one not-found case.

Complexity Analysis: State best-case, average-case, and worst-case time complexity.

Scenario-Based: Lost Product ID Search

Task 14: A store keeps recently scanned product IDs in an unsorted list. Search for a requested product ID and report whether it exists and at which position.

Requirements:

- Use at least 15 product IDs.
- Implement the search manually.
- Count and display comparisons.
- Test a target near the beginning and another near the end.

Complexity Analysis: Compare comparisons and relate them to O(1) best case and O(n) worst case.

Binary Search on Sorted Data

Simple Binary Search

Task 15: Create a sorted list of at least 15 integers and implement Binary Search to locate a target.

Requirements:

- Use low, high, and mid.
- Do not use built-in searching functions.
- Display the index when found.
- Display a not-found message otherwise.

Complexity Analysis: Explain why sorted order is required and analyze average/worst behavior as O(log n).

Scenario-Based: University SAP ID Lookup

Task 16: A university maintains student SAP IDs in sorted order. Write a Binary Search program that locates a requested SAP ID.

Requirements:

- Create at least 20 sorted SAP IDs.
- Ask the user for a target.
- Count comparisons.
- Test one existing and one non-existing SAP ID.

Complexity Analysis: Explain how sorted order allows half of the search space to be discarded after each comparison.

Compare Linear and Binary Search Complexity

Simple Search Comparison

Task 17: Using the same sorted list of at least 100 values, implement both Linear Search and Binary Search for the same target.

Requirements:

- Count comparisons for both.
- Display the result index from both.
- Test a target near the end.
- Create a small comparison table in output or report.

Complexity Analysis: Explain why Linear Search can require many more comparisons even though both algorithms return the same answer.

Scenario-Based: Search Strategy for a Large Student Database

Task 18: A university system contains many student records. Records may arrive unsorted from live registration or be maintained in sorted SAP ID order. Design and implement a program demonstrating the appropriate search strategy in each case.

Requirements:

- Use an unsorted dataset for Linear Search.
- Create a sorted version for Binary Search.
- Search for the same target.
- Measure execution time using time.perf_counter() and/or count comparisons.
- Discuss the requirement of maintaining sorted data.

Complexity Analysis: Provide a short conclusion comparing O(n) and O(log n), including why Binary Search depends on sorted order.

Aror University of Art, Architecture, Design & Heritage Sukkur.

Complexity Reference:

Operation	Typical Time	Reason	Auxiliary Space
Insert at end	O(1) amortized	No element shifting; occasional resizing	O(1) auxiliary
Insert at beginning	O(n)	Existing elements shift right	O(1) auxiliary
Insert in middle/specified position	O(n) worst case	Elements after position may shift right	O(1) auxiliary
Delete at end	O(1)	No remaining elements shift	O(1) auxiliary
Delete at beginning	O(n)	Remaining elements shift left	O(1) auxiliary
Delete in middle/specified position	O(n) worst case	Elements after position may shift left	O(1) auxiliary
Modify known index	O(1)	Direct indexed replacement	O(1)
Update all elements	O(n)	Each element is processed once	O(1) auxiliary

Execution-Time Measurement Note:

time.perf_counter() measures elapsed time using a high-resolution performance counter. The measured values may vary because of hardware, operating-system activity, Python version, and background processes. Use measured execution times to observe behavior, but determine Big-O complexity from the number of algorithmic operations, shifting, and direct accesses performed.

Submission Requirements:

- Save the Python program/notebook using the format: SAPID_YourName_Lab04.
- Write your SAP ID and Name at the beginning of the notebook.
- Complete all 10 lab exercises.
- Run all programs and include the output for each task.
- Use time.perf_counter() for the requested execution-time measurements.
- Add brief comments explaining important parts of the code.
- For every exercise, state and justify time complexity and auxiliary space complexity.
- Where positions are compared, explain how the number of shifted elements affects performance.
- Submit the completed lab according to the instructor's submission instructions.